Module 20: Integration

Contents

Module 20: Integration 1
Prerequisites: 2
Scenario 3
Practice: Creating the C# Wrapper class 3
Practice: Creating the D365 for Fin & Ops class 4

Prerequisites:

1. Open Dynamics 365 for Finance & Operations landing page

Scenario

An external web service will be called from Dynamics 365 for Fin & Ops through a C# wrapper class.

Practice: Creating the C# Wrapper class

1. Open Visual Studio and close the Existing Solution by clicking on File -> Close Solution
2. Click on File -> New Project
3. Select template Class Library under Visual C#
4. Type project name “AirportDataProvider”
5. Save the project in a new Solution “AirportService”
6. Right click on References and click on Add Service References
7. Type http://www.webservicex.net/airport.asmx?WSDL
8. Add AirportServRef as NameSpace and click OK
9. Add a new C# class AirportData.cs
10. Type the following code

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using AirportDataProvider.AirportServRef;
using System.ServiceModel;

namespace AirportDataProvider
{
    0 references
    public class AirportData
    {
        0 references
        public static string GetInfo(string code)
        {
            // This trivial sample doesn't load service address from the config file,
            // doesn't close the client and doesn't deal with exceptions.
            // You should do it in real implementations.

            var binding = new System.ServiceModel.BasicHttpBinding();
            var endpointAddress = new EndpointAddress("http://www.webservicex.net/airport.asmx");
            airportSoapClient client = new airportSoapClient(binding, endpointAddress);

            return client.getAirportInformationByAirportCode(code);
        }
    }
}
```

11. Build the Project AirportDataProvider

Practice: Creating the D365 for Fin & Ops class

1. In the Solution Explorer AirportService, right click and add a new Project
2. Select Dynamics 365 -> Unified Operations
3. Write project name: AirportService
4. Right click on Reference and select Add Reference
5. Click on browse and select AirportDataProvider.dll
6. In the Solution Explorer AirportService, right click and Add -> New Item
7. Under Operations Artifacts, Click Code
8. Select Class and create a new Runnable Class AirportTest
9. Within main method, add following code
   ```
   info(AirportDataProvider.AirportData::GetInfo('KUL'));
   ```
10. Build the solution and click on Start

Proprietary and Confidential
Subject to Change